Submission Worksheet

Submission Data

Course: IT202-007-F2025
Assignment: IT202 - Milestone 3
Student: Dylan G. (dg599)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 12/8/2025 6:35:09 PM
Updated: 12/8/2025 10:16:35 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-3/grading/dg599
View Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-3/view/dg599

Instructions

1. Refer to Milestone3 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone3 branch
    1. `git checkout Milestone3` (ensure proper starting branch)
    2. `git pull origin Milestone3` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On Github merge the pull request from `Milestone3` to `dev`
    5. On Github create a pull request from `dev` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout dev`
11. `git pull origin dev`

Section #1: (3 pts.) Api Data

Progress: 100%

Task #1 (1 pt.) - Concept of Data Association

Progress: 100%

Details:

- What's the concept of your data association to users? (examples: favorites, wish list, purchases, assignment, etc)
- Describe with a few sentences

Your Response:

The concept of my data association to users is called My Garage it is a personal car collection essentially. Users are able to associate cars with their user to create their own garage of that they are interested in or own. This concept of My Garage is like a favorites page. The associations is stored in the UserCars table in the database.

Saved: 12/8/2025 8:29:57 PM

Task #2 (1 pt.) - Data Updates

Progress: 100%

Details:

- When an associated entity is updated (manually or API) how is the association affected?
  - Does the user see the old version of the data?
  - Does the user see the new version of the data?
  - Does the user need to have data re-associated or remapped?
- Explain why.

Your Response:

So, when an associated entity is updated either manually or through the API the user will see the new verision of the data automatically. The user will not have to re-associate or remap the car to their garage, it will update automatically and be the newest data in their garage. How this work is that the UserCars table stores only the user_id and car_id. This means that it isn't storing a copy of the data. When the user looks at their garage the webpage perfroms a JOIN query, meaning that the query is always getting the newest data from the Cars table.

Saved: 12/8/2025 8:34:51 PM

Task #3 (1 pt.) - Handling Association

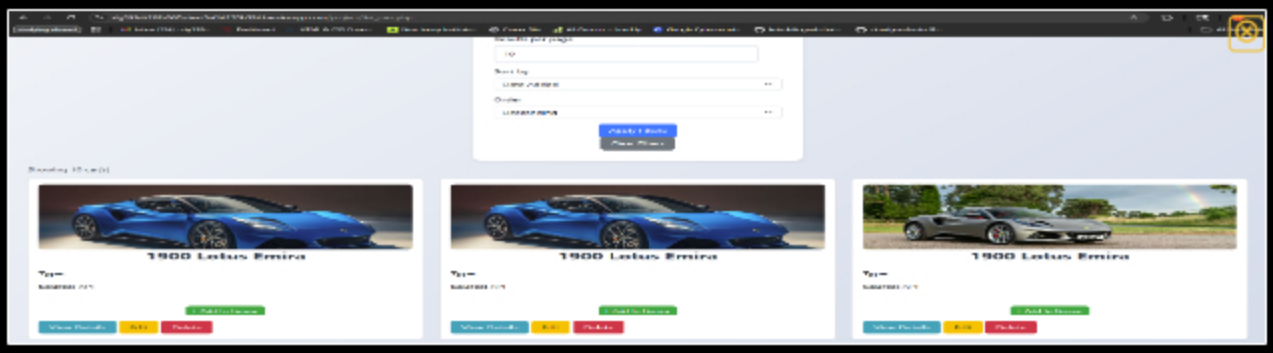
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show an example page of where a user can get data associated with them
- Ensure heroku dev url is visible
- Caption if this is a user-facing page or admin page
- Include screenshots of code related to this logic including the database query



shows the button that will add the car to your garage

add car to garage from view car

```php
// Get user's garage cars if logged in
$user_garage_ids = [];
if (is_logged_in()) {
    $garage_stmt = $db->prepare("SELECT car_id FROM UserCars WHERE user_id = :user_id");
    $garage_stmt->execute([":user_id" => get_user_id()]);
    $garage_results = $garage_stmt->fetchAll(PDO::FETCH_COLUMN);
    $user_garage_ids = $garage_results ?: [];
}
?>
```

Checks if car is in garage code

```php
<p><strong>Source:</strong> <?php echo (se($car, 'is_api', 0, false) == 1) ? " API" : " Manual"; ?></p>

<?php if (is_logged_in()): ?>
    <form method="POST" action="toggle_garage.php" style="margin-bottom: 0.5rem;">
        <input type="hidden" name="car_id" value="<?php echo se($car, 'id', ''); ?>" />
        <input type="hidden" name="redirect" value="list_cars.php?<?php echo http_build_query($_GET); ?>" />
        <?php if ($in_garage): ?>
            <input type="hidden" name="action" value="remove" />
            <button type="submit" class="btn btn-sm btn-danger">✓ In Garage</button>
        <?php else: ?>
            <input type="hidden" name="action" value="add" />
            <button type="submit" class="btn btn-sm btn-success">+ Add to Garage</button>
        <?php endif; ?>
    </form>
<?php endif; ?>
```

add to garage button code

```php
// Check if already in garage
$stmt = $db->prepare("SELECT id FROM UserCars WHERE user_id = :user_id AND car_id = :car_id LIMIT 1");
$stmt->execute([":user_id" => $user_id, ":car_id" => $car_id]);
$existing = $stmt->fetch(PDO::FETCH_ASSOC);

if ($action === "add") {
    if ($existing) {
        flash("This car is already in your garage", "warning");
    } else {
        $stmt = $db->prepare("INSERT INTO UserCars (user_id, car_id) VALUES (:user_id, :car_id)");
        try {
            $stmt->execute([":user_id" => $user_id, ":car_id" => $car_id]);
            flash("Car added to your garage!", "success");
        } catch (Exception $e) {
            flash("Error adding car to garage", "danger");
            error_log("Error adding to garage: " . var_export($e, true));
        }
    }
```

query to add car to garage

Saved: 12/8/2025 9:02:54 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature
- Include the heroku prod link to this page that triggers this logic and handles this logic

URL #1
https://github.com/dylangarca/dg599-IT202-007/pull/1

URL
https://github.com/dylangarca/dg

URL #2
https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/list_cars.php

URL
https://dg599-it202-007-prod-013

Saved: 12/8/2025 9:02:54 PM

Part 3:

Progress: 100%

Details:

- Describe the process of associating data with the user.
- Can it be toggled, or is it applied once?

Your Response:

The process of associating that car data to the user is with a toggle mechanism, so that means it can be added and removed many times. How it works is that the user will click the add to garage button, the form submits with the car_id and the action to add. It then checks if the association exists and if it doesn't it will insert a new row and add it into the garage. The way the removing works is the exact same the user will click the remove button and it will submit the form with the action to remove. After that it will remove the association but the car will still remain in the Cars table. This process allows for the same car to be added and removed to a user as many times as they like.

Saved: 12/8/2025 9:02:54 PM

Section #2: (6 pts.) Associations

Progress: 100%

Task #1 (1.50 pts.) - Logged-in User's Associated Entities

Progress: 100%

Details:

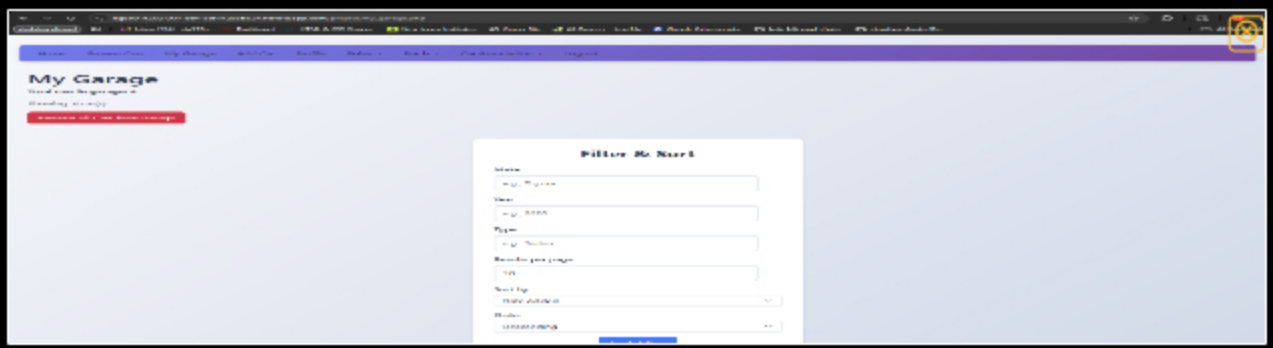
- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a link/button to remove all associations for the particular user
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
    - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
    - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

Details:

- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible
- Include the code screenshots related to this page



filter and sort of my garage

my garage cars

single car view from garage

```
                    <form method="POST" action="toggle_garage.php" style="display: inline;">
                        <input type="hidden" name="car_id" value="<?php echo se($car, 'id', ''); ?>" />
                        <input type="hidden" name="action" value="remove" />
                        <input type="hidden" name="redirect" value="my_garage.php?<?php echo http_build_query($_GET); ?>" />
                        <button type="submit" class="btn btn-danger" >Remove from Garage</button>
                    </form>
                </div>
            </div>
        </div>
```

remove one association

```
$stmt = $db->prepare("DELETE FROM UserCars WHERE user_id = :user_id");
try {
    $stmt->execute([":user_id" => $user_id]);
    $deleted_count = $stmt->rowCount();
    flash("Removed $deleted_count car(s) from your garage", "success");
} catch (Exception $e) {
    flash("Error removing cars from garage", "danger");
    error_log("Error removing all from garage: " . var_export($e, true));
}

```

remove all associations

query code

Saved: 12/8/2025 9:36:39 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature
- Include the heroku prod link to this page

URL #1
https://github.com/dylangarca/dg599-IT202-007/pull/51

URL
https://github.com/dylangarca/dg

URL #2
https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/my_garage.php

URL
https://dg599-it202-007-prod-013

Saved: 12/8/2025 9:36:39 PM

Part 3:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

The way that I solved the association output is with an inner join query with the Cars and UserCars table. So the code joins cars.id with usercars.car_id while filtering by the current users id. This make sure that only the cars that are associated with the logged in user are showed. For the line item requirements each card shows information that is fetched from the Cars table through JOIN. This data includes year, make, model, type and also has the date it was added. For the stats, i went with two stage counting, showing the total count and the displayed count. For the total count it runs a modified version of the main query the key part that is select cars with select count to being able to get the total number of cars in the garage. For the displayed count i use the count($cars) to show how many cars are currently showing. For the filter/sorting, the filter system is buuilding the SQL query The sorting is handled by validating the sort column against an allowed list.It validates the order direction as well.

Saved: 12/8/2025 9:36:39 PM

Task #2 (1.50 pts.) - All Users Association Page

Progress: 100%

Details:

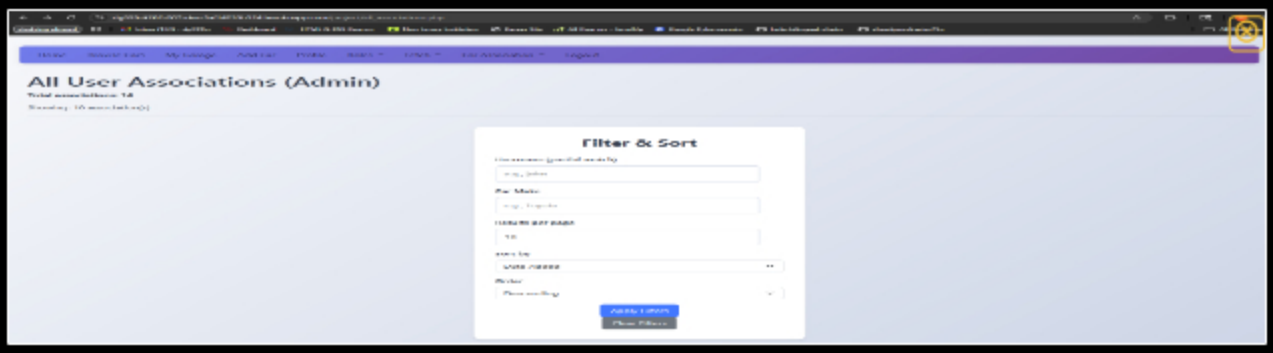
- Each line item should have a logical summary
- Each line item should include the username this entity is associated with
    - Clicking the username should redirect to that user's public profile
- Each line item should include a column that shows the total number of users the entity is associated with
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
    - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
    - A filter with no matching records should show "no results available" or equivalent

Part 1:

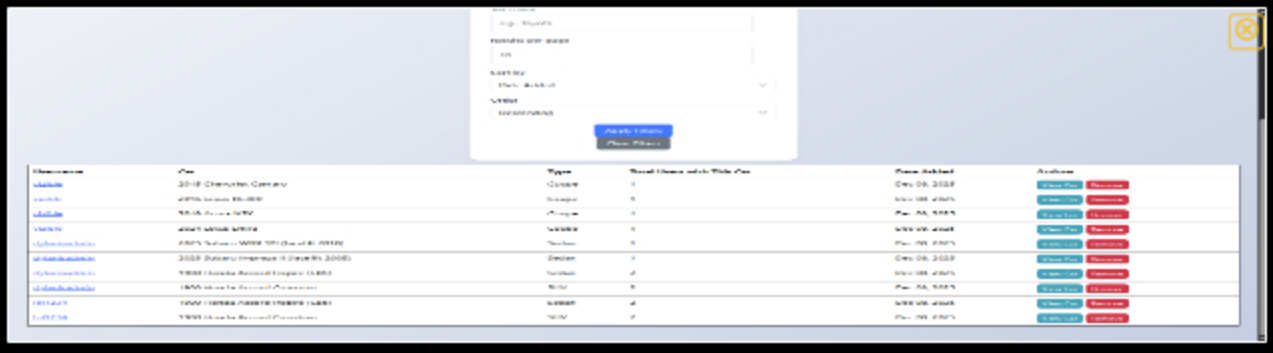
Progress: 100%

Details:

- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query



all associations filter and sort



all associations table with users and relationship

all associations user profile

all associations no results and full delete of all relationships

```php
$query = "SELECT UserCars.*, Cars.make, Cars.model, Cars.year, Cars.type, Cars.image_url, Users.username,
          (SELECT COUNT(*) FROM UserCars uc2 WHERE uc2.car_id = UserCars.car_id) as total_users_with_car
          FROM UserCars
          INNER JOIN Cars ON UserCars.car_id = Cars.id
          INNER JOIN Users ON UserCars.user_id = Users.id
          WHERE 1=1";
$params = [];

if (!empty($username_filter)) {
    $query .= " AND Users.username LIKE :username";
    $params[":username"] = "%" . $username_filter . "%";
}

if (!empty($make_filter)) {
    $query .= " AND Cars.make LIKE :make";
    $params[":make"] = "%" . $make_filter . "%";
}
```

query code

table code

```php
$car_stmt = $db->prepare("SELECT Cars.*, UserCars.created as added_date
                          FROM Cars
                          INNER JOIN UserCars ON Cars.id = UserCars.car_id
                          WHERE UserCars.user_id = :user_id");
$car_stmt->execute([":user_id" => $user["id"]]);
$cars = $car_stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
?>
```

user profile query

```
<?php if (empty($cars)): ?>
    <p>This user has no cars in their garage.</p>
<?php else: ?>
    <div class="row">
        <?php foreach ($cars as $car): ?>
            <div class="col-md-4 mb-3">
                <div class="card" style="border: 1px solid #ddd; border-radius: 0.5rem; padding: 1rem;">
                    <h3><?php echo se($car, 'year', ''); ?> <?php echo se($car, 'make', ''); ?> <?php echo se($car, 'model', ''); ?></h3>
                    <p><strong>Type:</strong> <?php echo se($car, 'type', 'N/A'); ?></p>
                    <p><small>Added: <?php
                        $added = se($car, 'added_date', '', false);
                        echo !empty($added) ? date('M d, Y', strtotime($added)) : 'Unknown';
                    ?></small></p>
                    <a href="view_car.php?id=<?php echo se($car, 'id', ''); ?>" class="btn btn-info">View Details</a>
                </div>
            </div>
        <?php endforeach; ?>
    </div>
<?php endif; ?>
```

user profile code

Saved: 12/8/2025 9:38:48 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature
- Include the heroku prod link to this page

URL #1
https://github.com/dylangarca/dg599-IT202-[illegible]749

URL
https://github.com/dylangarca/dg

URL #2
https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/all_associations.php

URL
https://dg599-it202-007-prod-013

URL #3
https://github.com/dylangarca/dg599-IT202-[illegible]763

URL
https://github.com/dylangarca/dg

Saved: 12/8/2025 9:38:48 PM

Part 3:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

The way that I solved the association output is with two inner join queries inner join Cars and inner join Users table. These get the car details and to get the username.This make sure that all the cars that are associated with a user are showed. For the line item requirements the table shows each association as a row and it contains the username with a clickable link that will send the user the users profile, it has the car information, has the total users with an association to that car, has the date the association was added, and has buttons to view or remove the car. For the stats, i went with two stage counting, showing the total count and the displayed count. For the total count it runs a modified version of the main query the key part is the select count, which gets the total number of associations. For the displayed count i use the count($associations) to show how many associations are currently showing. For the filter/sorting, the filter system is buuilding the SQL query. The username filter is the key differentiator for this page compared to the other filter forms.

Saved: 12/8/2025 9:38:48 PM

Task #3 (1.50 pts.) - Unassociated Page

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

Details:

- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

unassociated cars sort and filter, sections stats

unassociated cars no results

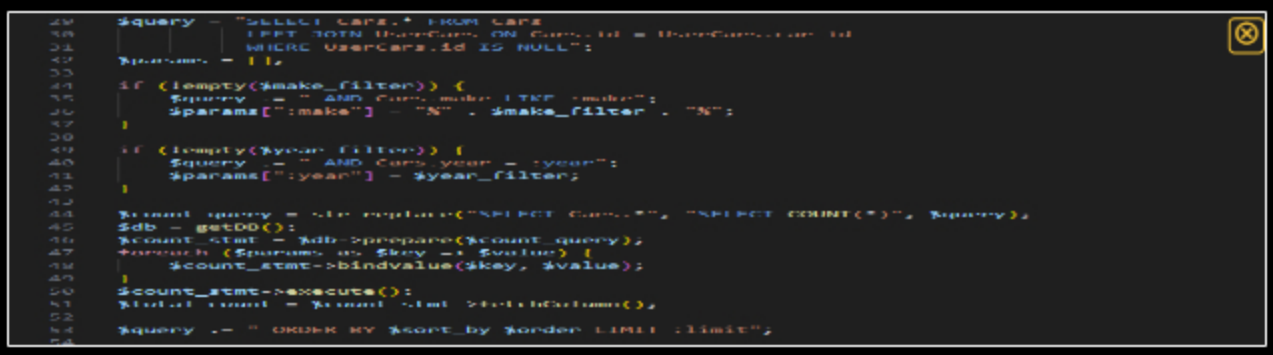
unassociated cars results

```
<?php if (empty($cars)): ?>
    <div class="alert alert-info">
        <strong>No results available.</strong> All cars have at least one user association!
    </div>
<?php else: ?>
    <div class="row">
        <?php foreach ($cars as $car): ?>
            <div class="col-md-4 mb-4">
                <div class="card" style="border: 1px solid #dddd; border-radius: 8.5rem; padding: 1rem;">
                    <?php if (!empty($car['image_url'])): ?>
                        <img src="<?php echo se($car, 'image_url', '', false); ?>"
                             alt="<?php echo se($car, 'make', ''); ?> <?php echo se($car, 'model', ''); ?>"
                             style="width: 100%; height: 200px; object-fit: cover; border-radius: 8.5rem; background: #f0f0f0;" />
                    <?php else: ?>
                        <div style="width: 100%; height: 200px; background: #eee; display: flex; align-items: center; justify-content: center; border-radius: 8.5rem;">
                            No Image
                        </div>
                    <?php endif; ?>

                    <h5><?php echo se($car, 'year', ''); ?> <?php echo se($car, 'make', ''); ?> <?php echo se($car, 'model', ''); ?></h5>
                    <p><strong>Type:</strong> <?php echo se($car, 'type', 'N/A'); ?></p>

                    <div class="mt-2">
                        <a href="view_car.php?id=<?php echo se($car, 'id', ''); ?>" class="btn btn-info">View Details</a>
                    </div>
                </div>
            </div>
        <?php endforeach; ?>
    </div>
<?php endif; ?>
```

unassociated cards code



```
    $query = "SELECT Cars.* FROM Cars
              LEFT JOIN UserCars ON Cars.id = UserCars.car_id
              WHERE UserCars.id IS NULL";
    $params = [];

    if (!empty($make_filter)) {
        $query .= " AND Cars.make LIKE :make";
        $params[":make"] = "%" . $make_filter . "%";
    }

    if (!empty($year_filter)) {
        $query .= " AND Cars.year = :year";
        $params[":year"] = $year_filter;
    }

    $count_query = str_replace("SELECT Cars.*", "SELECT COUNT(*)", $query);
    $db = getDB();
    $count_stmt = $db->prepare($count_query);
    foreach ($params as $key => $value) {
        $count_stmt->bindValue($key, $value);
    }
    $count_stmt->execute();
    $total = $count_stmt->fetchColumn();

    $query .= " ORDER BY $sort_by $order LIMIT :limit";
```

unassociated query

Saved: 12/8/2025 9:49:50 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature
- Include the heroku prod link to this page

URL #1
https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/unassociated_cars.php

URL
https://dg599-it202-007-prod-013

URL #2
https://github.com/dylangarca/dg599-IT202-007

URL
https://github.com/dylangarca/dg

Saved: 12/8/2025 9:49:50 PM

Part 3:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

The way that I solved the association output is with the left join query with the null check. The left join will retunr all the cars and tries to match them with a Usercars entry, if the a car has a association the usescars.id will have a value so if there isnt an association the usercars.id will be null and that is where the null check comes in to play. For the line item requirements cars are shown in a card grid layout showing the car image, year, make, modeal, wehicle type, and has a view details button. The card layout is better than the table because the cards are more visually appealing for this project. For the stats, i went with two stage counting, showing the total count and the displayed count. For the total count it runs a modified version of the main query the key part is the select count, which gets the total number of cars. For the displayed count i use the count($cars) to show how many cars are currently showing. For the filter/sorting, the filters are applied directly to the cars table using the make and year filter. Sorting is limited to created, make, and year.

Saved: 12/8/2025 9:49:50 PM

Task #4 (1.50 pts.) - Admin Association Page (Like User Roles)

Progress: 100%

Details:

- The page should have a form with two fields
    - Partial match for username
    - Partial match for entity reference (name or something user-friendly)
- Submitting the form should give up to 25 matches of each
    - Likely best to show as two separate columns
- Each entity and user will have a checkbox next to them
- Submitting the checked associations should apply the association if it doesn't exist; otherwise it should remove the association
    - A filter with no matching records should show "no results available" or equivalent

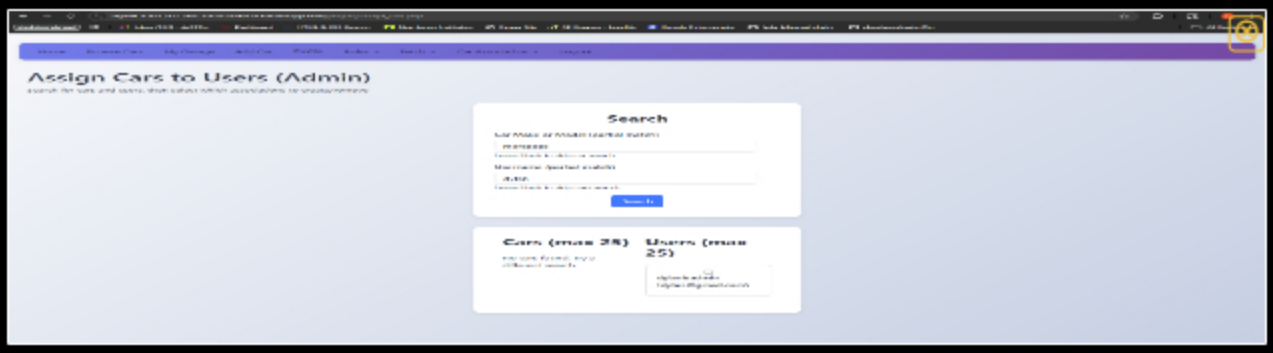
Part 1:

Progress: 100%

Details:

- Show a few examples of this page from heroku dev with various selections having been submitted
- Ensure heroku dev url is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

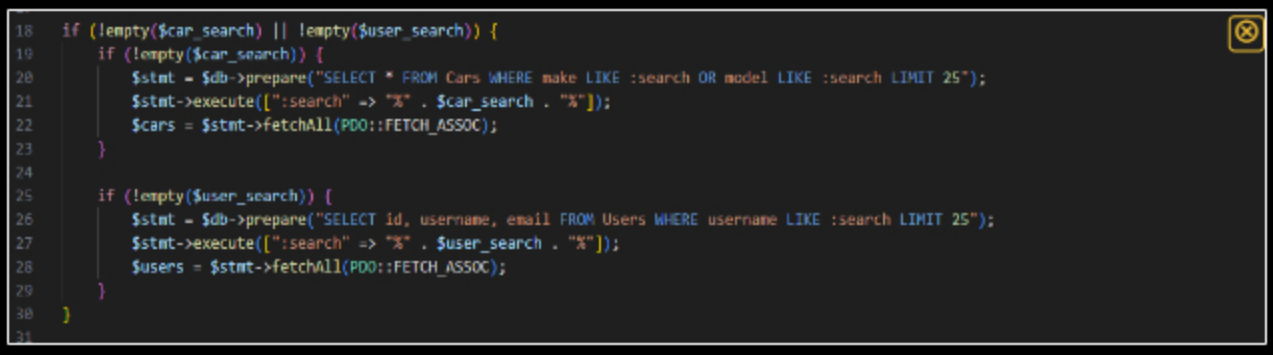
assign cars filter and sort



assign cars no result

assign cars successful addition

assign cars association code



```php
if (!empty($car_search) || !empty($user_search)) {
    if (!empty($car_search)) {
        $stmt = $db->prepare("SELECT * FROM Cars WHERE make LIKE :search OR model LIKE :search LIMIT 25");
        $stmt->execute([":search" => "%" . $car_search . "%"]);
        $cars = $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    if (!empty($user_search)) {
        $stmt = $db->prepare("SELECT id, username, email FROM Users WHERE username LIKE :search LIMIT 25");
        $stmt->execute([":search" => "%" . $user_search . "%"]);
        $users = $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

assign cars query

assign car checkbox code

Saved: 12/8/2025 9:59:31 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature
- Include the heroku prod link to this page

URL #1

URL

https://dg500-it202-007-prod-013

https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/assign_cars.php

URL #2

https://github.com/dylangarca/dg599-IT202-007

URL

https://github.com/dylangarca/dg

Saved: 12/8/2025 9:59:31 PM

Part 3:

Progress: 100%

Details:

- Describe how your code solves the requirements; be clear

Your Response:

The code solves the requirements with the two search fields with partial matching. The form has two search inputs that use LIKE queries they are the car search and the user search. They both use LIKE '%search%' to be able to find matches anywhere in the text. Both of the queries are also limited to 25 results. I used bootstrap to create a responsive tow coloum layout to show the matching cars and the matching users. So each item is also rendering in as a bootstrap checkbox.

Saved: 12/8/2025 9:59:31 PM

Section #3: (1 pt.) Misc

Progress: 100%

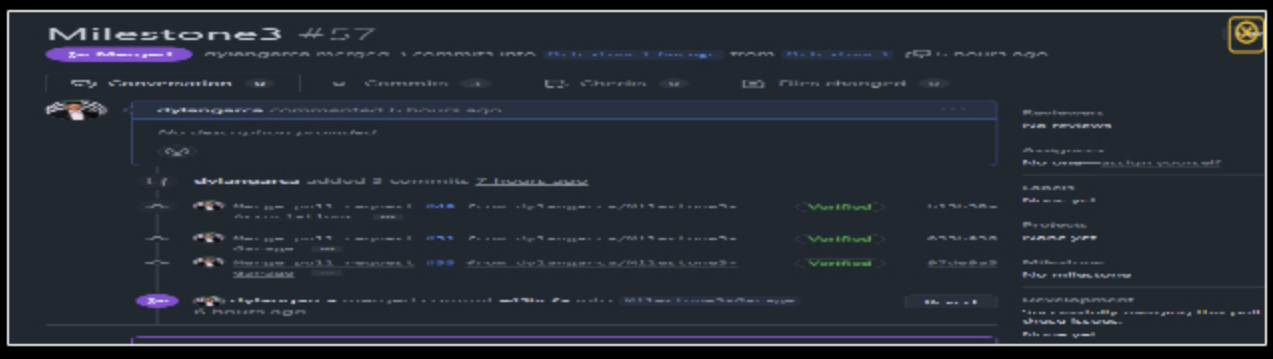
Task #1 (0.33 pts.) - Github Details

Progress: 100%

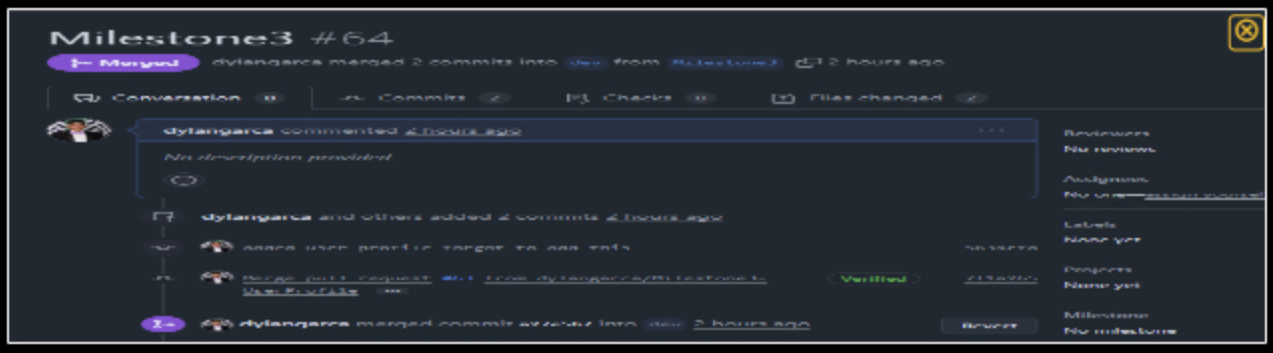
Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



commits.p1



commits p2

Saved: 12/8/2025 10:03:07 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request for Milestone3 to dev (should end in `/pull/#`)

URL #1
https://github.com/dylangarca/dg599-IT202-007/pull/64
URL
https://github.com/dylangarca/dg

URL #2
https://github.com/dylangarca/dg599-IT202-007/pull/67
URL
https://github.com/dylangarca/dg

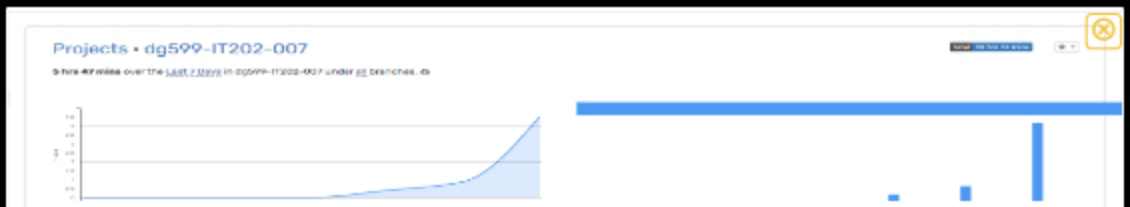
Saved: 12/8/2025 10:03:07 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



wakatime p1

wakatime p2

Saved: 12/8/2025 10:04:47 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned alot from this milestone, espcially from the sql queries and the association pages. I found the assign cars page to be quite intersting because of the addition of user profiles. I orignally came in thinking that this was just a my garage type of web app but with the addition of user profiles in assign cars it puts into perspective that this could be turned into a social media type of web app where you can show of your garage to your friends.

Saved: 12/8/2025 10:13:07 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think the easiest part was the filter/sort form. This is beacause once you have completed one filter/sort form you are able to use the form to create the other filter/sort forms you just need to change them a little bit depending on the page. Another part that was fairly easy was the creation of all the action buttons such as the add and remove buttons. I would also saying chaning the nav bar to the one with bootstrap was quite easy but tedious to go in a change each link and name.

Saved: 12/8/2025 10:10:59 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment for me would have had to be getting the bootstrap and css to fully work properly the way i wanted it to. To be specific i had a form css in my stylesheet and in certain buttons it would create a white background border around it Another part. So I had trouble taking it off because i didnt notice that the style sheet was overiding the bootstrap. So what I did to solve this was i added inline bootstrap/css to remove the background color and the border it was creating from the form css class in the stylesheet. I would also say parts of the association pages were quite diffucult to get correct and how i wanted them.

Saved: 12/8/2025 10:16:35 PM